

辞書 型

超サポ
愉快カンパニー

アシスト

QAシート

- 気になったことについては気軽にQAシートに記入してください
 - https://docs.google.com/spreadsheets/d/1MxqWSRFn6r3BCzAa17TtrWa_HoRhxp0aUoN6Su5KDb0/edit?usp=drive_link

作成したEC2インスタンスを確認

i-05b748c60ff44d0be (p04-yamamoto10152-Linux-ssh) のインスタンス概要 情報

接続 インスタンスの状態 ▼ アクション ▼

less than a minute 前に更新済み

インスタンス ID i-05b748c60ff44d0be	パブリック IPv4 アドレス [REDACTED]	プライベート IPv4 アドレス [REDACTED]
IPv6 アドレス [REDACTED]	インスタンスの状態 ① 保留中	DNS [REDACTED]
ホスト名のタイプ IP 名: ip-10-0-5-29.ap-northeast-1.compute.internal	プライベート IP DNS 名 (IPv4 のみ) ip-10-0-5-29.ap-northeast-1.compute.internal	
プライベートリソースの DNS 名に応答 -	インスタンスタイプ t2.medium	Elastic IP アドレス [REDACTED]
自動的に割り当てられた IP アドレス -	VPC ID vpc-02243d68ba98a0677 (p04-yamamoto10152-vpc-01) ↗	AWS Compute Optimizer の検出結果 このインスタンスで利用可能な推奨オプションはありません。
IAM ロール -	サブネット ID subnet-077652917c2ed2e46 (p04-subnet-public1) ↗	Auto Scaling グループ名 [REDACTED]
IMDSv2 Required	インスタンス ARN arn:aws:ec2:ap-northeast-1:958533308904:instance/i-05b748c60ff44d0be	管理対象 False
オペレーター -		

パブリックIPアドレスを確認してメモしておく

このIPアドレスは毎朝変わるため
このフェーズの間は毎朝確認して
設定ファイルを変更する

辞書とは？～名前でデータを管理する箱～

Key(名前)とValue(値)のペアでデータを管理します

辞書とは？

Key(名前)とValue(値)の
ペアでデータを管理する箱

例：社員情報

```
{"name": "山田",  
  "age": 25,  
  "dept": "営業部"}
```

日常例で考えると

 英和辞典のイメージ

apple → りんご

banana → バナナ

orange → みかん

単語 (Key) を引くと
意味 (Value) が出てくる！



Keyという名前でValueを取り出せる！
番号ではなく意味のある名前でアクセスできるのが辞書の特徴

リストとの違い～どういうときに使うか～

番号より名前でアクセスしたいとき → 辞書の出番！

リスト（番号でアクセス）

```
person = ["山田", 25, "営業部"]
```

何番目か覚えておく必要がある

```
person[0] → 山田
```

```
person[1] → 25
```

```
person[2] → 営業部
```

😓 0番が名前か年齢かわかりにくい...

辞書（名前でアクセス）

```
person = {  
    "name": "山田",  
    "age": 25,  
    "dept": "営業部"  
}
```

```
person["name"] → 山田
```

```
person["age"] → 25
```

😊 名前で取り出せてわかりやすい！

辞書の作り方 (Key・Value)

{ } を使ってKey: Valueのペアを定義します

```
# 辞書の作り方: { } で囲み、Key: Value の形で書く
person = {
    "name": "山田",      # Key: "name"  Value: "山田"
    "age": 25,          # Key: "age"   Value: 25
    "dept": "営業部"    # Key: "dept"  Value: "営業部"
}
```

```
# Keyは文字列が一般的。Valueはどんな型でもOK
scores = {"math": 80, "english": 95, "science": 72}
```

書き方のルール

- { } で全体を囲む
- Key: Value をカンマで区切る
- Keyは通常 "" で囲む文字列
- Valueはどんな型でもOK

空の辞書も作れる

```
empty = { }
```

後から要素を追加できる
→ slide6で詳しく説明

Keyで値を取り出す～何が便利なのか～

辞書[Key] でValueを取り出せます。意味のある名前でアクセスできるのが強みです

```
person = {"name": "山田", "age": 25, "dept": "営業部"}
```

```
# Keyを [ ] で指定してValueを取り出す
print(person["name"])    # → 山田
print(person["age"])     # → 25
print(person["dept"])    # → 営業部
```

存在しないKeyはエラー

```
person["email"]
→ KeyError !
```

辞書にないKeyを指定すると
エラーになる

→ inで確認してから
アクセスするのが安全

f文字列と組み合わせると

```
print(f'名前：{person["name"]}')
→ 名前：山田
```

```
print(f'年齢：{person["age"]}歳')
→ 年齢：25歳
```

読みやすいコードが書ける！

辞書の追加・更新

辞書[Key] = Value の形で追加・更新ができます

```
person = {"name": "山田", "age": 25}

# 要素の追加（存在しないKeyに代入）
person["dept"] = "営業部"
print(person)
# → {"name": "山田", "age": 25, "dept": "営業部"}

# 要素の更新（存在するKeyに再代入）
person["age"] = 26
print(person)
# → {"name": "山田", "age": 26, "dept": "営業部"}
```



追加も更新も書き方は同じ！Keyが存在しなければ追加、存在すれば上書きされる

keys() / values() / items()

辞書の中身をまとめて取り出すときに使います。特に items()はfor文と組み合わせると便利です

```
person = {"name": "山田", "age": 25, "dept": "営業部"}

# keys() : Keyだけ取り出す
print(person.keys())    # → dict_keys(['name', 'age', 'dept'])

# values() : Valueだけ取り出す
print(person.values())  # → dict_values(['山田', 25, '営業部'])

# items() : KeyとValueをセットで取り出す (for文と相性抜群！)
for key, value in person.items():
    print(f"{key} : {value}")
# → name : 山田  age : 25  dept : 営業部
```



items()はDay2で学んだfor文と組み合わせると全要素を順番に処理できる！

やってみよう:辞書型

自分のプロフィールを辞書で作って操作してみましょう

① 自分のプロフィールを辞書で作ってみよう

```
profile = {  
    "name": "〇〇",  
    "age": 〇〇,  
    "hobby": "〇〇"  
}
```

② Keyを指定して情報を取り出してみよう

```
print(f'名前: {profile["name"]}')  
print(f'趣味: {profile["hobby"]}')
```

③ 新しい項目を追加してみよう

```
profile["dept"] = "〇〇部"  
print(profile)
```

④ items()とfor文で全項目を表示してみよう

```
for key, value in profile.items():  
    print(f'{key}: {value}')
```

まとめ

辞書とは

Key（名前）とValue（値）のペアでデータを管理。{ } で作り、辞書[Key]でValueを取り出す

リストとの使い分け

番号より名前でアクセスしたいとき→ 辞書。
意味のあるKeyをつけることでコードが読みやすくなる

追加・更新

辞書[Key] = Value で追加・更新。Keyが存在しなければ追加、存在すれば上書き

items()とfor文

for key, value in 辞書.items(): で全要素を順番に処理できる